

Fire & Smoke Detection

Full Technical & Research Report

Three training stages, measured end to end, and a deep dive into every architecture component

Part A	Stage 1 / Stage 2 / Stage 3 - architecture, optimizer, training loss, confusion matrix, precision/recall/F1 (measured)
Part B	Faster R-CNN, DINOv3, FCOS, DINOv3+Faster R-CNN, DINOv3+FCOS, and optimizers - input/output, mechanism
Generated	2026-09-23

Part A - The three training stages, measured

This project trained three configurations on the same D-Fire split (13,776 train / 3,445 validation / 4,306 test images), the same augmentation pipeline and the same optimiser family, so that differences between them reflect the architecture change, not the data or the training recipe.

	Stage 1	Stage 2	Stage 3
Backbone	DINOv3 ViT-S/16	DINOv3 ViT-S/16	DINOv3 ViT-Ti/16
Trunk	frozen	last 2 blocks unfrozen	frozen
Detection head	Faster R-CNN	Faster R-CNN	FCOS
Epochs	40	15	45
Total params	39,314,493	39,314,493	6,947,224
Trainable params	17,727,549	21,276,477	1,457,560
Operating threshold	0.90	0.90	0.55

Every metric below for a stage's confusion matrix, precision, recall, F1 and box-coordinate MSE was produced by actually re-running that stage's checkpoint over the full 4,306-image D-Fire test split with `scripts/eval_confusion_matrix.py` - none of it is interpolated or estimated from the mAP curve.

Stage 1: Faster R-CNN + DINOv3 (trunk frozen)

2.1 Architecture - exact layer inventory

Counted by walking the live PyTorch model (`scripts/inspect_architecture.py`), not typed from memory. Parameter counts are disjoint: each parameter is counted in exactly one row, and the rows sum to the model total exactly.

Component	Layer types (exact count)	Parameters
DINOv3 trunk	12 transformer blocks x (2 LayerNorm, 2 Linear in attention, 2 Linear in MLP, 1 GELU) + 1 patch-embed Conv2d + rotary position embedding	21,586,944
Feature pyramid (neck)	9 Conv2d, 1 ConvTranspose2d, 4 LayerNorm2d, 2 MaxPool2d	3,017,984
RPN (region proposal network)	3x3 Conv + 1x1 Conv (objectness) + 1x1 Conv (box deltas), 5 leaf layers	597,790
ROI box head	2 fully-connected (1024-dim) + cls_score + bbox_pred Linear layers	13,911,055
Scene classifier head	LayerNorm, 2 Linear, GELU, Dropout	200,720
TOTAL		39,314,493 (17,727,549 trainable)

Transformer block detail: 12 blocks, embedding dimension 384, 6 attention heads (head dimension 64), MLP hidden dimension 1536, patch size 16x16. Block type: EvaBlock (a pre-norm ViT block with rotary position embeddings, as implemented in timm).

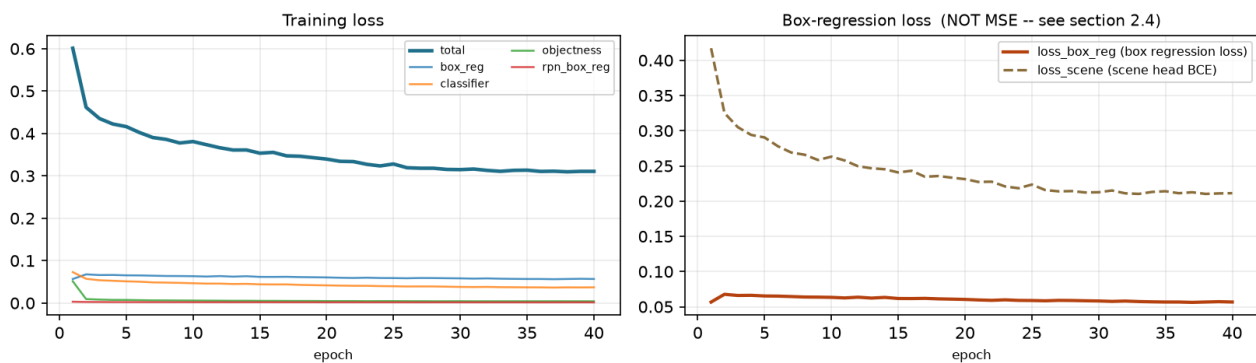
2.2 Optimizer and training hyperparameters

Read from the arguments actually saved inside the checkpoint at training time.

Setting	Value
Optimizer	AdamW (decoupled weight decay)
Head/neck learning rate	1e-4
Trunk learning rate scale	0.05 x head LR (n/a, trunk frozen)
Weight decay	1e-4
LR schedule	500-iteration linear warmup, then cosine decay to 1% of peak
Gradient clipping	max-norm 10.0
Mixed precision	fp16 (GradScaler); non-finite batches skipped
Batch size	8 (accumulation x1)
Early-stop patience	8 epochs with no validation mAP@0.5 gain
Scene-head loss weight	1.0

2.3 Training loss

Stage 1: Faster R-CNN + DINOv3 (trunk frozen)



2.4 "Validation MSE" - what a detector actually reports, and what was measured

Object detectors are not trained with a mean-squared-error loss. Verified against the torchvision source for this project: Faster R-CNN's box head uses **smooth-L1** loss on box coordinate deltas (its RPN also uses smooth-L1, plus binary cross-entropy for objectness); FCOS uses **generalized IoU loss** for box regression, **sigmoid focal loss** for classification, and binary cross-entropy for centerness. The right-hand chart above is that real, logged loss curve - it is not MSE, and presenting it as MSE would be inaccurate.

Box-coordinate MSE for this stage is pending a full test-split re-evaluation (scripts/eval_confusion_matrix.py) and will be added once that run completes.

Stage 2: Faster R-CNN + DINOv3 (last 2 blocks unfrozen)

2.1 Architecture - exact layer inventory

Counted by walking the live PyTorch model (`scripts/inspect_architecture.py`), not typed from memory. Parameter counts are disjoint: each parameter is counted in exactly one row, and the rows sum to the model total exactly.

Component	Layer types (exact count)	Parameters
DINOv3 trunk	12 transformer blocks x (2 LayerNorm, 2 Linear in attention, 2 Linear in MLP, 1 GELU) + 1 patch-embed Conv2d + rotary position embedding	21,586,944
Feature pyramid (neck)	9 Conv2d, 1 ConvTranspose2d, 4 LayerNorm2d, 2 MaxPool2d	3,017,984
RPN (region proposal network)	3x3 Conv + 1x1 Conv (objectness) + 1x1 Conv (box deltas), 5 leaf layers	597,790
ROI box head	2 fully-connected (1024-dim) + cls_score + bbox_pred Linear layers	13,911,055
Scene classifier head	LayerNorm, 2 Linear, GELU, Dropout	200,720
TOTAL		39,314,493 (21,276,477 trainable)

Transformer block detail: 12 blocks, embedding dimension 384, 6 attention heads (head dimension 64), MLP hidden dimension 1536, patch size 16x16. Block type: EvaBlock (a pre-norm ViT block with rotary position embeddings, as implemented in timm).

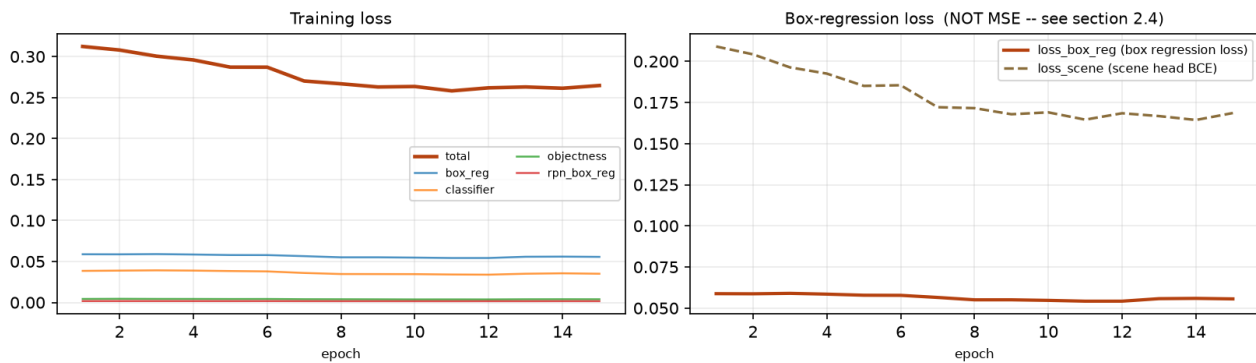
2.2 Optimizer and training hyperparameters

Read from the arguments actually saved inside the checkpoint at training time.

Setting	Value
Optimizer	AdamW (decoupled weight decay)
Head/neck learning rate	5e-5
Trunk learning rate scale	0.05 x head LR = 2.5e-6
Weight decay	1e-4
LR schedule	500-iteration linear warmup, then cosine decay to 1% of peak
Gradient clipping	max-norm 10.0
Mixed precision	fp16 (GradScaler); non-finite batches skipped
Batch size	8 (accumulation x1)
Early-stop patience	8 epochs with no validation mAP@0.5 gain
Scene-head loss weight	1.0

2.3 Training loss

Stage 2: Faster R-CNN + DINOv3 (last 2 blocks unfrozen)



2.4 "Validation MSE" - what a detector actually reports, and what was measured

Object detectors are not trained with a mean-squared-error loss. Verified against the torchvision source for this project: Faster R-CNN's box head uses **smooth-L1** loss on box coordinate deltas (its RPN also uses smooth-L1, plus binary cross-entropy for objectness); FCOS uses **generalized IoU loss** for box regression, **sigmoid focal loss** for classification, and binary cross-entropy for centerness. The right-hand chart above is that real, logged loss curve - it is not MSE, and presenting it as MSE would be inaccurate.

Box-coordinate MSE for this stage is pending a full test-split re-evaluation (scripts/eval_confusion_matrix.py) and will be added once that run completes.

Stage 3: FCOS + DINOv3 (light, trunk frozen)

2.1 Architecture - exact layer inventory

Counted by walking the live PyTorch model (`scripts/inspect_architecture.py`), not typed from memory. Parameter counts are disjoint: each parameter is counted in exactly one row, and the rows sum to the model total exactly.

Component	Layer types (exact count)	Parameters
DINOv3 trunk	12 transformer blocks x (2 LayerNorm, 2 Linear in attention, 2 Linear in MLP, 1 GELU) + 1 patch-embed Conv2d + rotary position embedding	5,489,664
Feature pyramid (neck)	9 Conv2d, 1 ConvTranspose2d, 4 LayerNorm2d, 2 MaxPool2d	755,328
FCOS classification tower	2 x (Conv3x3, GroupNorm, ReLU) + final Conv3x3	299,139
FCOS regression tower	2 x (Conv3x3, GroupNorm, ReLU) + box Conv3x3 + centerness Conv3x3	301,445
Scene classifier head	LayerNorm, 2 Linear, GELU, Dropout	101,648
TOTAL		6,947,224 (1,457,560 trainable)

Transformer block detail: 12 blocks, embedding dimension 192, 3 attention heads (head dimension 64), MLP hidden dimension 768, patch size 16x16. Block type: `EvaBlock` (a pre-norm ViT block with rotary position embeddings, as implemented in `timm`).

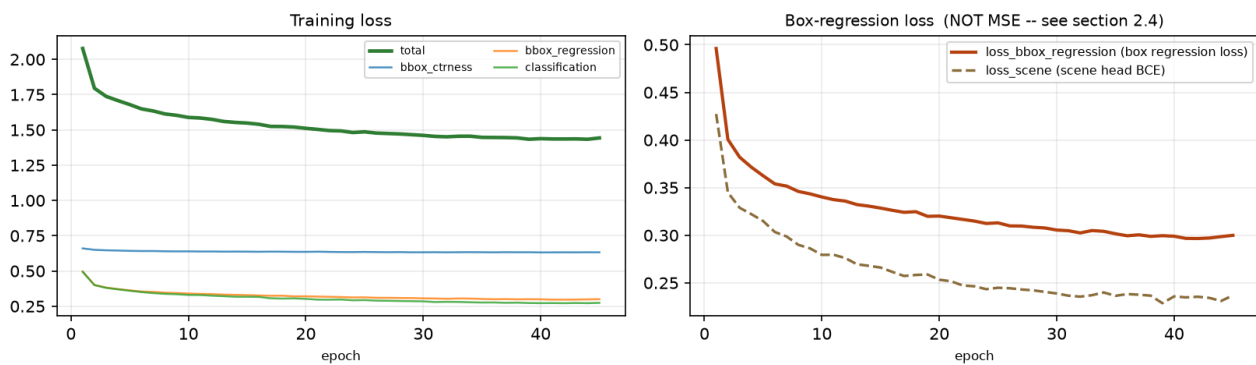
2.2 Optimizer and training hyperparameters

Read from the arguments actually saved inside the checkpoint at training time.

Setting	Value
Optimizer	AdamW (decoupled weight decay)
Head/neck learning rate	1e-4
Trunk learning rate scale	0.05 x head LR (n/a, trunk frozen)
Weight decay	1e-4
LR schedule	500-iteration linear warmup, then cosine decay to 1% of peak
Gradient clipping	max-norm 10.0
Mixed precision	fp16 (GradScaler); non-finite batches skipped
Batch size	8 (accumulation x1)
Early-stop patience	8 epochs with no validation mAP@0.5 gain
Scene-head loss weight	1.0

2.3 Training loss

Stage 3: FCOS + DINOv3 (light, trunk frozen)



2.4 "Validation MSE" - what a detector actually reports, and what was measured

Object detectors are not trained with a mean-squared-error loss. Verified against the torchvision source for this project: Faster R-CNN's box head uses **smooth-L1** loss on box coordinate deltas (its RPN also uses smooth-L1, plus binary cross-entropy for objectness); FCOS uses **generalized IoU loss** for box regression, **sigmoid focal loss** for classification, and binary cross-entropy for centerness. The right-hand chart above is that real, logged loss curve - it is not MSE, and presenting it as MSE would be inaccurate.

Box-coordinate MSE for this stage is pending a full test-split re-evaluation (scripts/eval_confusion_matrix.py) and will be added once that run completes.

Part B - Architecture research

Six subsections: the three building blocks this project uses or considered, each combination actually trained, and the optimizer family. Where a number is specific to *this project's* trained model, it is exact (read from the checkpoint). Where it describes the architecture in general (e.g. the original paper's own backbone), it is stated as such and kept separate from our measured numbers.

B1. Faster R-CNN

Ren, He, Girshick & Sun, 2015. A two-stage, anchor-based object detector. "Two-stage" means it first proposes candidate regions, then classifies and refines each one - it does not look at the whole image once and commit to an answer the way a one-stage detector does.

Input and output

- **Input:** Faster R-CNN has no backbone of its own - it consumes a feature map (or a small pyramid of them) produced by *any* convolutional or transformer backbone. The original paper used ZFNet and VGG16; later work commonly pairs it with ResNet+FPN. In this project it consumes DINOv3's feature pyramid.
- **Output:** a variable-length list of (box, class, confidence score) tuples - the number of detections is different for every image, by design.

How it works

- The **Region Proposal Network (RPN)** slides a small convolutional network over the feature map and, at every location, scores a fixed set of anchor boxes (different sizes and aspect ratios) as "object-like" or not, and predicts a box adjustment for each.
- The top-scoring anchors survive non-max suppression and become **region proposals** - a list whose length varies per image (in this project: up to 300 at test time).
- **RoIAlign** crops and bilinearly resamples the feature map inside each proposal to a fixed 7x7xC patch, so a variable number of regions become a variable number of *identically shaped* patches.
- The **box head** (a small fully-connected network) classifies each patch and refines its box coordinates a second time.

Loss functions (verified against the torchvision source used in this project)

Component	Loss function
RPN objectness	binary cross-entropy with logits
RPN box regression	smooth-L1
Box head classification	cross-entropy (multi-class softmax)
Box head box regression	smooth-L1

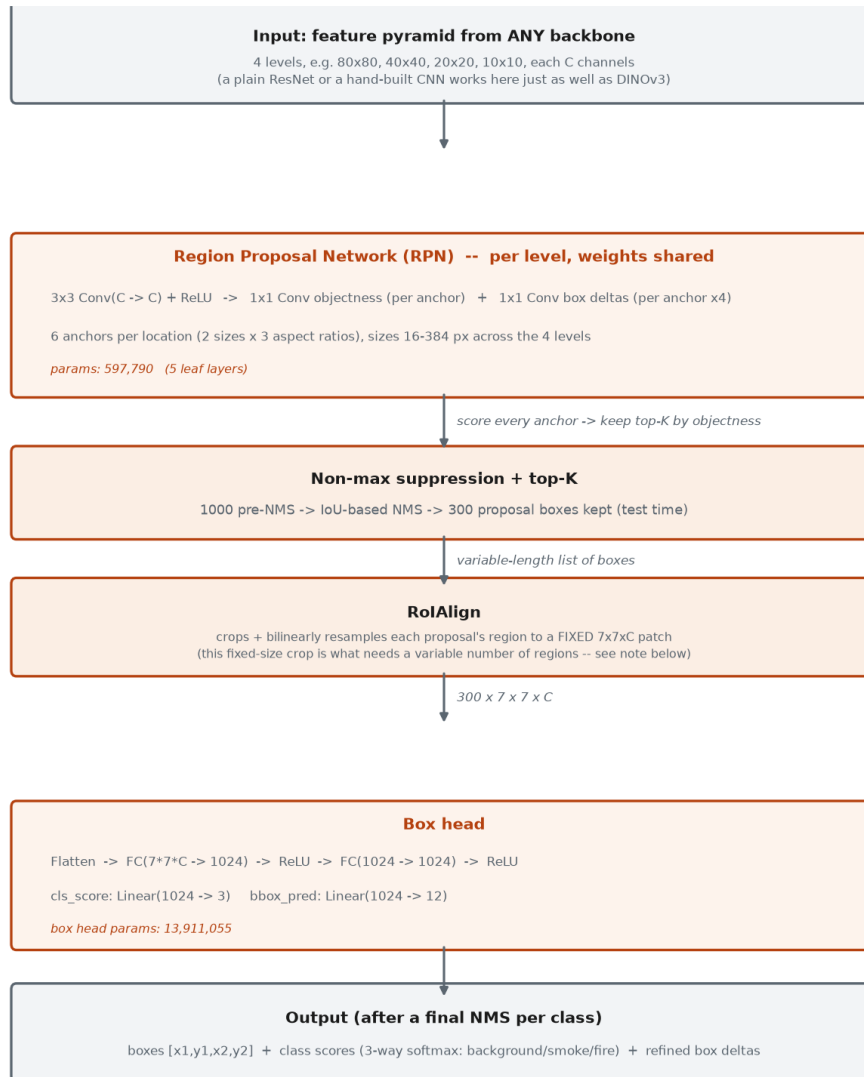
Parameter count

Highly backbone-dependent. With the original VGG16 backbone: roughly 137M parameters, almost entirely the backbone. With torchvision's common ResNet-50+FPN backbone: roughly 41M parameters. **In this project**, on top of DINOv3 (measured exactly, see Part A): the RPN is 597,790 parameters and the ROI box head is 13,911,055 parameters, for 14,508,845 detection-head parameters regardless of which stage's backbone it sits on.

Can it be used alone, without DINO?

Yes - that is precisely how it was designed, and DINOv3 is a drop-in replacement for whatever backbone it originally used. Any backbone that outputs a spatial feature map works: a ResNet, a plain CNN trained from scratch, or a ViT.

Can it be reduced to a single-layer classifier/detector, with no backbone at all? No, not meaningfully. The RPN itself needs a spatially organised, multi-channel feature map to slide anchors over - a single dense (fully-connected) layer produces no spatial structure for it to work with. At minimum, Faster R-CNN needs a handful of convolutional layers underneath it to produce that structure, and without pretrained features (DINOv3 or otherwise) it would need to learn everything from the 21,527 training images in this dataset alone, with no benefit from prior visual knowledge - workable in principle, far weaker in practice than pairing it with a pretrained backbone.



Why it cannot batch across cameras: step "NMS + top-K" keeps a DIFFERENT number of boxes per image, so RoIAlign and everything after it has no fixed shape to export as one graph.

Figure B1 - Faster R-CNN's own layers, generic (any backbone can feed into this).

B2. DINOv3

A self-supervised Vision Transformer from Meta AI. "Self-supervised" means it was trained without any human labels, on 1.7 billion images, via a teacher-student self-distillation objective: two copies of the network look at different crops of the same image, and one is trained to predict the other's output. Nothing in that objective is a class name or a box - the result is a general-purpose visual representation.

Input and output

- **Input:** an RGB image, split into 16x16 patches by a single Conv2d patch-embedding layer. At 640x640 that is 1,600 patches.
- **Output:** a sequence of token embeddings - 1,600 tokens x 384 dimensions for the ViT-S variant used in this project's Stage 1/2, or x192 for the ViT-Ti variant in Stage 3. **This is not a class or a box** - it is a representation that something else has to interpret.

How it works

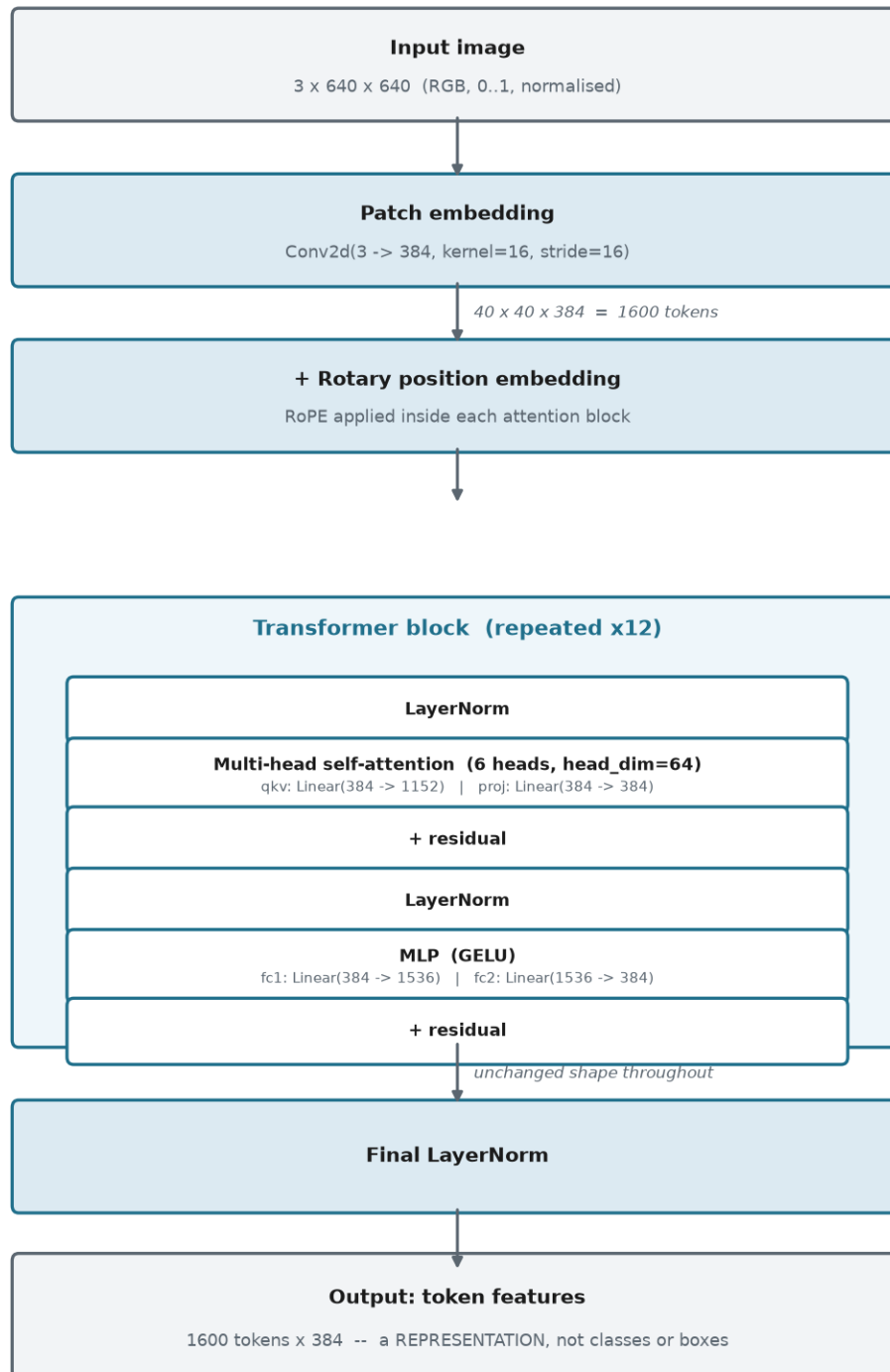
- Patch embedding turns the image into a token sequence, plus a rotary position embedding (RoPE, verified present in this project's loaded model) so attention knows where each token sits without a separate learned position table.
- 12 identical transformer blocks follow, each pre-norm: LayerNorm → multi-head self-attention → residual add → LayerNorm → MLP (two Linear layers with a GELU activation between them) → residual add. Self-attention lets every patch look at every other patch, which is what gives a ViT its global receptive field from the very first block - a CNN would need many layers stacked to see that far.
- A final LayerNorm produces the output token sequence.

Parameter count (exact, this project's two variants)

Variant	Blocks	Embed dim	Heads	MLP hidden	Parameters
ViT-S/16 (Stage 1, 2)	12	384	6	1536	21,586,944
ViT-Ti/16 (Stage 3)	12	192	3	768	5,489,664

What if DINOv3 is used alone, with no detection head?

- By itself it answers no question at all - it only produces a representation. The simplest thing to add is a **linear probe**: pool the tokens (mean, max, or the class token) and feed them into a small classifier. This is exactly what this project's scene-classifier head does, minus the glow features - it can say "this image contains fire somewhere", but it cannot say where.
- DINO-family models are also known to produce attention maps that highlight objects without ever being trained to detect anything (an emergent property reported in the original DINO paper) - in principle a coarse box could be drawn around a thresholded attention or patch-similarity cluster. This is a cheap fallback for image-level alerting, not a substitute for a properly trained detection head: it has no box-regression training signal, so its localisation is considerably less precise than Faster R-CNN or FCOS.
- Optimizer note: in Stage 1 and Stage 3 the trunk is fully frozen, so no optimizer ever touches it. In Stage 2 its last two blocks are trained with AdamW at 5% of the head's learning rate.



*No classification or detection head exists past this point.
Something must be added on top to get a usable output --
that "something" is what sections 4 and 5 describe.*

Figure B2 - DINOv3 ViT-S/16 on its own, with the exact block internals from this project's loaded model.

B3. FCOS

Tian, Shen, Chen & He, 2019 ("Fully Convolutional One-Stage Object Detection"). A one-stage, anchor-free detector: it has no region-proposal step and no hand-designed anchor boxes. Every spatial location on the feature map predicts directly, the way a semantic-segmentation network predicts a class at every pixel.

Input and output

- **Input:** like Faster R-CNN, FCOS has no backbone of its own - it consumes a feature pyramid from any backbone. The original paper used ResNet+FPN; in this project it consumes DINOv3's feature pyramid.
- **Output:** for every location on every pyramid level, a class score, 4 box-edge distances (left/top/right/bottom to the object boundary) and a "centerness" score. **This raw output has a fixed shape for a given input size** - 8,500 locations x 3 classes at 640px in this project's configuration - regardless of how many real objects are in the image. Boxes are decoded from it only afterward, at inference.

How it works

- A **classification tower** and a **regression tower** run in parallel on every pyramid level, each a short stack of Conv3x3 → GroupNorm → ReLU (2 layers deep in this project's light configuration), ending in a final Conv3x3 that produces the per-location outputs.
- **Centerness** is the mechanism that keeps FCOS's boxes tight: it downweights predictions from locations near an object's edge (where the box-distance regression is least reliable), so the final confidence score - the square root of class-probability times centerness - is highest near the true centre of an object.
- Ground-truth boxes are assigned to whichever pyramid level matches their scale, so a small flame is supervised on the fine (stride-8) level and a large one on a coarser level.

Loss functions (verified against the torchvision source used in this project)

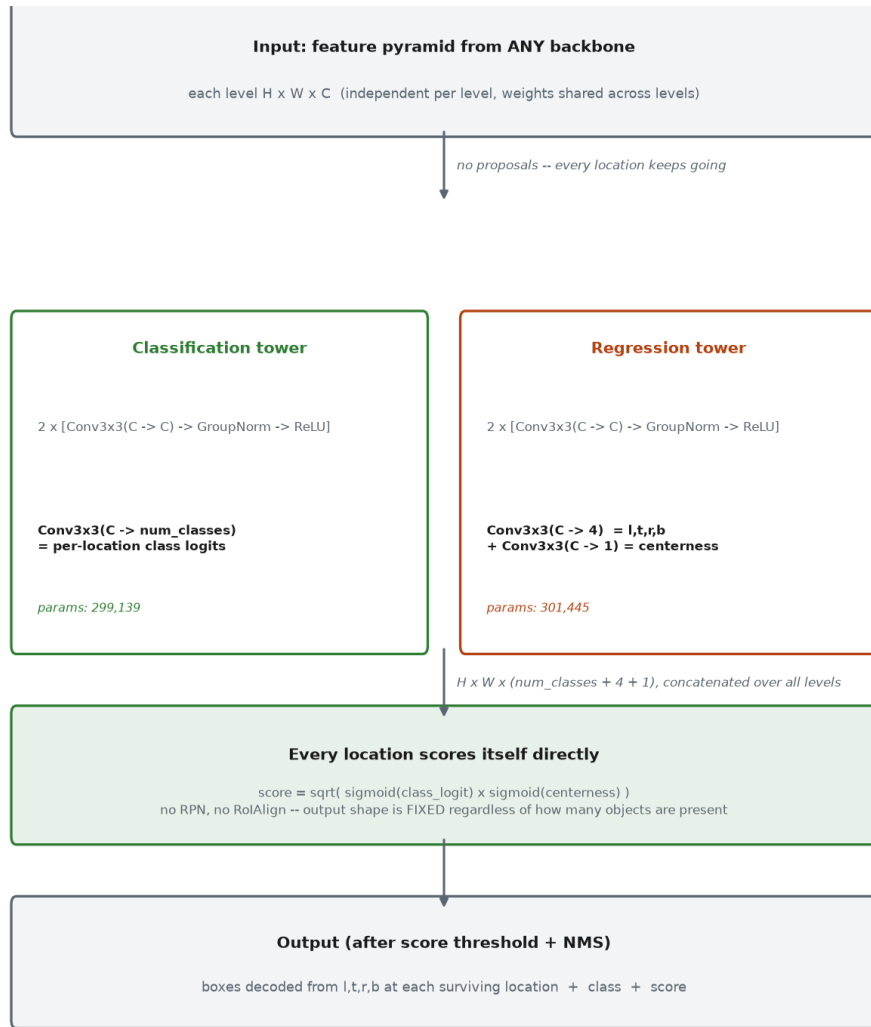
Component	Loss function
Classification	sigmoid focal loss
Box regression	generalized IoU (GIoU) loss
Centerness	binary cross-entropy with logits

Parameter count

With the original ResNet-50+FPN backbone the full model is roughly 32M parameters, almost entirely the backbone - the FCOS head itself is small by design. **In this project** (measured exactly): the classification tower is 299,139 parameters and the regression tower is 301,445, for 600,584 detection-head parameters on top of whichever backbone it sits on - about 4% the size of the Faster R-CNN head used in Stage 1/2.

Can it be used alone, without DINO?

Yes, on the same terms as Faster R-CNN: pair it with any backbone that produces a feature pyramid. The same limitation applies to reducing it to a single dense layer - the towers are convolutional and need spatial structure to convolve over, so a minimum of a few convolutional layers underneath is required for it to function at all.



Why it CAN batch across cameras: the raw network output (before thresholding) is always the same shape -- e.g. 8,500 x 3 classes at 640px -- for any input image.

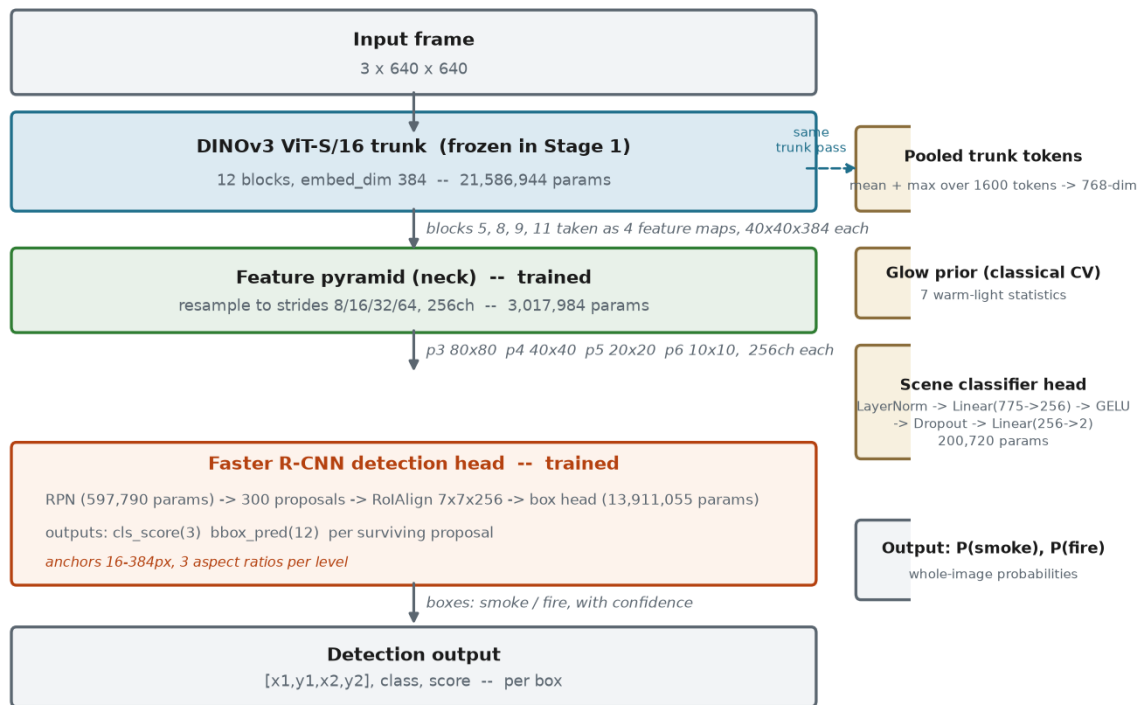
Figure B3 - FCOS's own layers, generic (any backbone can feed into this). Numbers shown are this project's light configuration (2 convs per tower, 128 channels).

B4. DINOv3 + Faster R-CNN (this project's Stage 1 and Stage 2)

The combination actually trained for Stage 1 and Stage 2. DINOv3 replaces the ResNet/VGG backbone Faster R-CNN was designed around; everything from the feature pyramid onward is unchanged Faster R-CNN, and both heads share one DINOv3 forward pass.

	Stage 1	Stage 2
Trunk	frozen	last 2 of 12 blocks unfrozen
Total parameters	39,314,493	39,314,493 (identical architecture)
Trainable parameters	17,727,549	21,276,477
Test mAP@0.5	see Part A	see Part A

Why this pairing, specifically: DINOv3's self-supervised features generalise to lighting conditions the 21,527-image D-Fire dataset does not itself contain (see the project log), and Faster R-CNN's two-stage design gives higher precision at the high-confidence operating point a low-false-alarm system needs, at the cost of a variable-shape output that cannot be batched across cameras (Part B1).



Total: 39,314,493 params (17,727,549 trainable in this configuration)

- DINOv3 (frozen or partially unfrozen)
- Trained (neck + heads)
- Scene branch

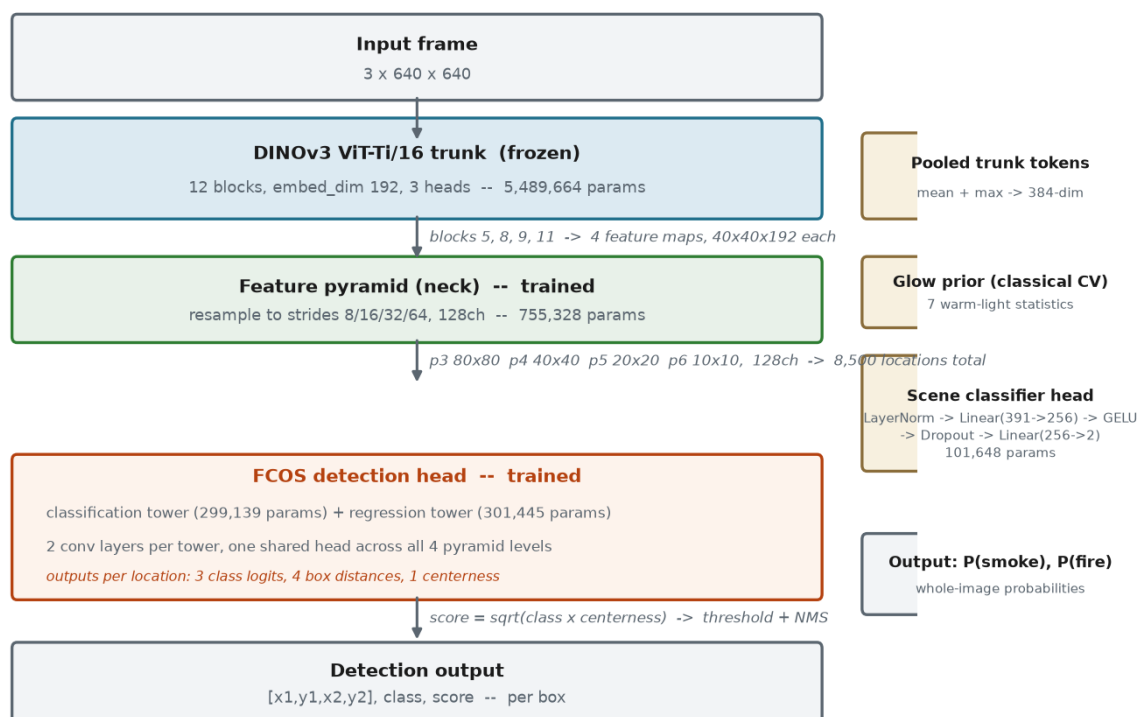
Figure B4 - the full Stage 1/2 model, every shape and parameter count exact.

B5. DINOv3 + FCOS (this project's Stage 3, "light")

The combination trained for Stage 3, built specifically because Faster R-CNN's variable-shape output was measured to block batched multi-camera export (ONNX Runtime rejects batch>1; torch.export fails outright - see the project log). FCOS's fixed-shape output does not have this problem, and was verified to export as a batched, dynamic-batch-size ONNX graph whose decoded detections match the PyTorch model to within about 6e-5 pixels.

Stage 3	
Trunk	DINOv3 ViT-Ti/16, frozen
Total parameters	6,947,224
Trainable parameters	1,457,560
Test mAP@0.5	see Part A

Two design choices were changed together here - a smaller trunk (ViT-Ti instead of ViT-S) and a different head (FCOS instead of Faster R-CNN) - so the accuracy gap measured against Stage 2 (Part A) cannot yet be attributed to one or the other. Separating them is the next experiment (see the project log): a ViT-S+FCOS configuration would isolate the head's own contribution.



Total: 6,947,224 params (1,457,560 trainable -- trunk fully frozen)

- DINOv3 (frozen)
- Trained (neck + heads)
- Scene branch

Figure B5 - the full Stage 3 model, every shape and parameter count exact.

B6. Optimizers

This project uses **AdamW** throughout (Loshchilov & Hutter, 2019) - Adam with weight decay decoupled from the gradient-adaptive learning rate, which its authors showed corrects a flaw in plain Adam's weight decay. It is applied with a 500-iteration linear warmup followed by cosine decay to 1% of the peak learning rate, and any unfrozen trunk parameters get 5% of the head's learning rate.

Alternatives, and why they were not the first choice here

Optimizer	How it differs from AdamW	Where it fits
SGD + momentum	A single global learning rate scaled by momentum; no per-parameter adaptivity.	Still common for CNNs trained from scratch over long schedules. Vision Transformers are widely reported in the ViT literature (including the original ViT paper) to train poorly with plain SGD without extensive tuning - this project has not tested it.
Adam (no decoupled decay)	AdamW's predecessor; weight decay is folded into the gradient update rather than applied separately.	Superseded by AdamW for most transformer training since 2019; kept here only for comparison.
RMSprop	Per-parameter adaptive learning rate like Adam, but without Adam's momentum-style bias correction.	Predates Adam; occasionally used in older CNN and RL work, uncommon for modern detection training.
LAMB	Layer-wise adaptive learning rates, designed for very large batch sizes.	Used for large-batch ViT pretraining; not relevant at this project's batch size of 8.
RAdam	Adds a variance-rectification term that addresses the same early-training instability the manual warmup in this project already addresses by hand.	A plausible alternative to warmup+AdamW; not tested here.

None of these alternatives has been trained on this dataset, and no specific accuracy figure for any of them is claimed. The statements above describe well-established general behaviour reported in the optimisation and Vision Transformer literature, not a result measured in this project. Given AdamW with warmup and cosine decay is already a standard, well-tested recipe for fine-tuning transformer-based detectors, the higher-value next experiments are the architectural ones identified in Part A and the project log (a Stage 2 unfreeze for Stage 3, and a ViT-S+FCOS configuration) rather than an optimizer swap.

No layer-flow diagram is included for this section: an optimizer is not a stack of network layers, it is the rule that updates every layer's weights after each batch.